PART 1.1 :- Auto encoders for anomaly detection.

Tasks :-

1. Calculate the total number of trainable parameters in the autoencoder, including weights and bias. Show your detailed steps for each layer.

Given Autoencoder Architecture :-

Input layer : 1000 neurons

Hidden layer 1 : FC, 512 neurons, ReLU

Bottleneck laye : FC, 32 neurons

Hidden layer 2 : FC, 512 neurons, ReLU

Output layer : FC, 1000 neurons, Sigmoid

Loss function : MSE

Regularization : L2 Regularization

Calculating Trainable Parameters :-

formula for Parameters in a layer is

$$\text{Total Parameters} = \underbrace{\text{Input size} \times \text{Output size}}_{\text{weights}} + \underbrace{\text{output size}}_{\text{biases}}$$

1. Input layer to hidden layer 1

Weights = 1000 × 512 = 512000

Biase = 512 (∵ one per neuron)

Total = 512000 + 512

= 512512

2. Hidden Layer 1 to Bottle neck layer :-

Weights = 512 × 32 = 16384

Biases = 32

Total parameters = 16384 + 32
= 16416
= 16416

3. Bottleneck layer to Hidden Layer 2 :-

Weights = 32 × 512 = 16384

Biases = 512

Total = 16384 + 512 = 16896

4. Hidden layer 2 to output layer :-

Weights = 512 × 1000 = 512000

Biases = 1000

Total = 512000 + 1000 = 513000

∴ Total Trainable parameters

= 512512 + 16416 + 16896 + 513000

= 1058824 paramters

2. Explain how L2 regularization term will be incorporated into the training process with MSE loss. Describe its impact on the weights updates during backpropagation.

Original MSE Loss $L_{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$

If we add L2 regularization to the above MSE then the total loss function becomes $L_{MSE} + L_{reg}$.

$$L_{reg} = \lambda \sum_i (w_i)^2 = \lambda \sum_i w_i^2$$

where λ is the regularization strength

w_i = model weights.

Total loss function $L_{Total} = L_{MSE} + L_{reg}$

Impact on weight updates during backpropagation :-

During the training process the weights are updated based on gradient of the Loss. When we consider L2 regularization, the gradient of loss function w.r.t to weights includes an extra term.

$$\frac{\partial \text{Total loss}}{\partial w} = \frac{\partial MSE}{\partial w} + 2\lambda w = \frac{\partial L_{MSE}}{\partial w} + 2\lambda w$$

This shrinks weights during updates thereby preventing them from growing too large.

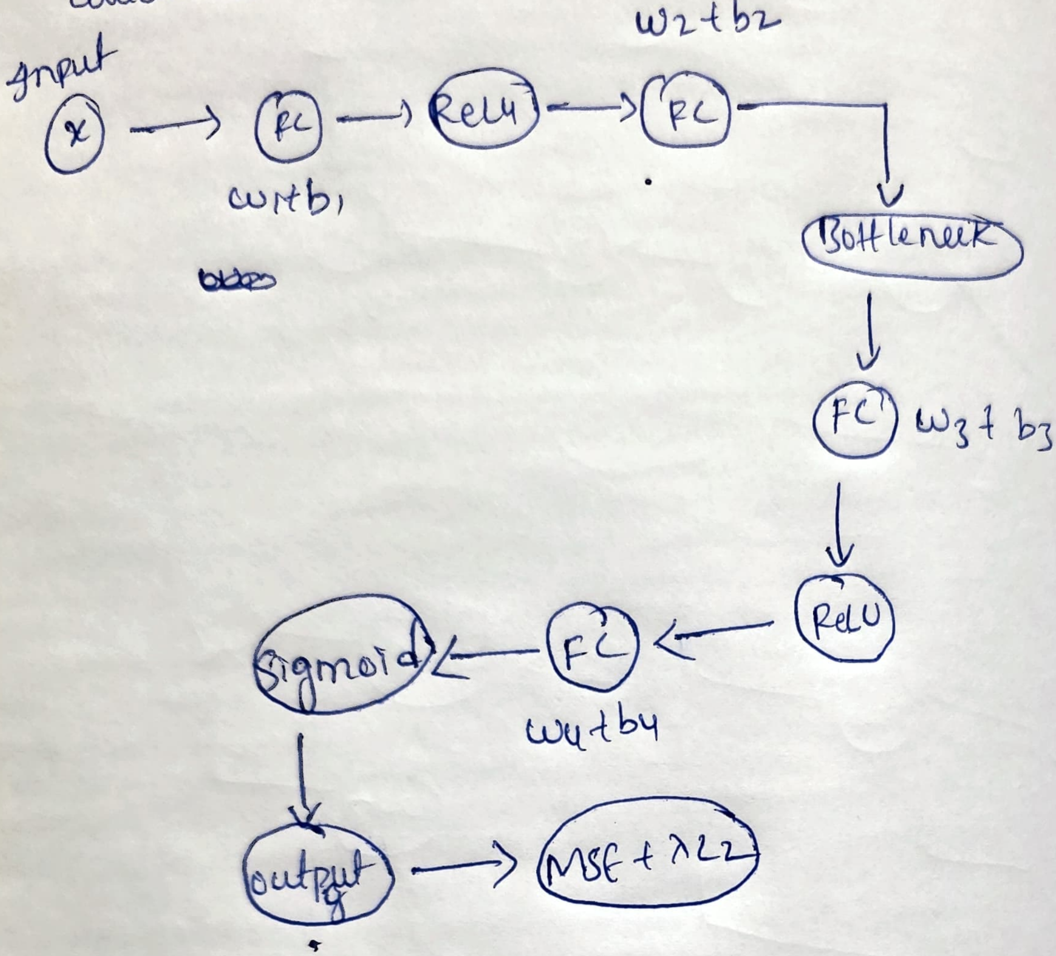
The updated weight formula becomes

$$W_{new} = W_{old} - \eta \left(\frac{\partial L_{MSE}}{\partial w} + 2\lambda w \right)$$

where η = learning rate.

This helps ~~to~~ prevent overfitting by discouraging the model from relying on complex weight patterns.

3. Draw a computational graph for the autoencoder



4) challenges of using Autoencoders for Anomaly Detection:-

* Training Data Requirements :- It requires large dataset of normal samples for training. If there are anomalies in training data, the model may learn to reconstruct them.

* overfitting :- Autoencoders are prone to overfitting if the model's architecture is too complex and if the amount of training data is limited.

* Difficulty in threshold selection :- There is no clear method for setting reconstruction error threshold for flagging anomalies.

* Subtle Anomalies :- These are small deviations from normal behavior which may not significantly impact the reconstruction loss making them difficult for autoencoders to detect reliably.

Limitations of using Autoencoders for Anomaly Detection :-

* Unsupervised Nature :- Autoencoders are operated without labelled data. So It is difficult for them to measure the accuracy or to evaluate performance against known anomalies.

* Assumes Abnormalies have high Reconstruction Error :-
Some anomalies may construct well if they resemble the normal data but fails for subtle anomalies that do not significantly distort reconstructions.

* Senstivity to Hyperparameters :- Autoencoder performance is highly dependent on selection of parameters. Finding optimal hyperparameters requires extensive experimentation.

* Computational cost :- Training autoencoders on large datasets can be computationally expensive. It is a significant limitation in real-time applications when dealing with high-dimentional data.

5. Propose Potential improvements or extensions to the System to enhance its effectiveness in detecting anomalies. consider architecture, loss, preprocessing, integration

Architecture Improvements :-

* using CNNs instead of FC layer is better when dealing with Spatial or temporal sensor data.
* Variational Autoencoders are useful in learning probabilistic representations and improves anomaly detection.

Loss function enhancements :-

* Instead of using MSE, we can use huberloss or SmoothL1 loss which are less sensitive to outliers and they also provide more stable gradients.

Data Preprocessing :-

* Robust scaling handles Sensor noise better. Generating synthetic anomalies can help in better threshold tuning.

System integration :-

* using dynamic thresholding helps in adusting the anomaly threshold based on distribution or adaptive methods. Applying sliding average with standard deviation windows and combining autoencoder output with traditional statistics can increase robustness and reduce false positives.